

EXECUTIVE SUMMARY

Target: http://isolated-1.com
Scan ID: TORTURE-P3-1-d4e49d74
Scan Date: 2026-03-04 17:31:04
Scan Duration: N/A
Total Requests Sent: 21
Avg Latency: N/A
Peak Concurrency: N/A
Findings: 2 vulnerabilities detected

Severity Breakdown: Critical: 0 | High: 2 | Medium: 0 | Low: 0
AI Inference Calls: 0
LLM Avg Latency: N/A
Circuit Breaker Activations: 0

- High severity vulnerabilities identified in system components
- Immediate action required to patch and secure affected areas
- Long-term focus on comprehensive security monitoring and updates
- Overall moderate risk level with priority given to high-severity issues

EXECUTIVE STRATEGIC IMPACT

```
{
  "Strategic_Attack_Path_Analysis": [
    "An adversary could exploit the Insecure Direct Object Reference vulnerability by manipulating direct object references in requests, allowing unauthorized access to sensitive data or functionality.",
    "The Prompt Injection flaw enables an attacker to inject malicious input into prompts used for automated interactions with systems, potentially leading to command execution or privilege escalation through crafted inputs.",
    "By combining these vulnerabilities, the attacker could craft a series of carefully constructed requests that first bypass IDOR protections by manipulating object references, then leverage prompt injection to execute privileged commands on compromised endpoints, culminating in high-impact system compromise."
  ]
}
```

DETAILED FINDINGS

FILTER: OBJECT REFERENCES (IDOR)

Finding #1: Insecure Direct Object Reference (IDOR)

CRITICAL

CWE: CWE-639
CVSS Score: 9.6 (CRITICAL)

THREAT SCORE: 96/100

Description:

- Insecure direct object reference in API endpoint allows unauthorized access to sensitive data.
- Attackers can manipulate URL parameters to retrieve unintended information or perform actions.
- Lack of proper input validation enables malicious users to exploit the vulnerability.

Impact:

- Unauthorized access to confidential user data and system resources.
- Potential for privilege escalation leading to full account takeover.
- Data breaches exposing sensitive customer information, resulting in reputational damage.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'IDOR' was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: GET | Param: param_0
URL: http://test-target-torture-.com/api/v1/endpoint_0
Analysis: The application failed to properly validate and sanitize user input, allowing arbitrary data to be passed in the endpoint parameter.

Table 1: Payload Decomposition

Component	Value	Technical Function
param_0	test_payload_0	Direct reference to target resource.
Access Check	Missing	Application fails to verify authorization.
Result	200 OK	Server returns data for unauthorized request.

Payload Specifications:

Vector Category: Insecure Direct Object Reference (IDOR)
Raw Payload: test_payload_0
Encoded: 746573745f7061796c6f61645f30
Encoding Type: None

Reproduction Command:

```
curl -X GET 'http://test-target-torture-.com/api/v1/endpoint_0'
```

HTTP Traffic Snapshot:

Request:

GET http://test-target-torture-.com/api/v1/endpoint_0 HTTP/1.1
Host: target
{}

Response:

HTTP/1.1 200 OK
Content-Type: application/json

{"status": "vulnerable", "data": "revealed"}

Remediation:

- - Implement proper input validation and sanitization of all URL parameters.
- - Use secure object identifiers that are not directly accessible via URLs.
- - Enforce strict access controls at the application level to restrict unauthorized actions.

Recommended Code Fix:

```
// Example fix for IDOR vulnerability in Java Spring Boot app
@GetMapping("/api/v1/endpoint_0")
public ResponseEntity<String> getEndpoint(@RequestParam String id) {
    // Validate and sanitize input before accessing data
    if (isValidId(id)) {
        return ResponseEntity.ok("Data retrieved successfully.");
    }
    throw new AccessDeniedException();
}
```

FILTER: INJECTION & FUZZING

Finding #2: AI Prompt Injection

CRITICAL

CWE: CWE-77
CVSS Score: 9.0 (CRITICAL)

THREAT SCORE: 90/100

Description:

- The endpoint is vulnerable to prompt injection due to lack of input validation.
- Attackers can manipulate the request payload to inject malicious commands or data.
- Successful exploitation could lead to unauthorized actions, data leakage, and system compromise.

Impact:

- Unauthorized command execution leading to potential privilege escalation.
- Data exposure through leaked sensitive information in response payloads.
- System instability or denial of service due to injected code execution.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'PROMPT_INJECTION' was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: PUT | Param: param_1
URL: http://test-target-torture-.com/api/v1/endpoint_1
Analysis: The application failed to properly validate and sanitize user input, allowing malicious payloads to be injected into API requests.

Table 1: Payload Decomposition

Component	Value	Technical Function
param_1	test_payload_1	Direct reference to target resource.
Access Check	Missing	Application fails to verify authorization.
Result	200 OK	Server returns data for unauthorized request.

Payload Specifications:

Vector Category: AI Prompt Injection

Raw Payload: test_payload_1

Encoded: 746573745f7061796c6f61645f31

Encoding Type: None

Reproduction Command:

```
curl -X PUT 'http://test-target-torture-.com/api/v1/endpoint_1'
```

HTTP Traffic Snapshot:

Request:

PUT http://test-target-torture-.com/api/v1/endpoint_1 HTTP/1.1

Host: target

{}

Response:

HTTP/1.1 200 OK

Content-Type: application/json

{"status": "vulnerable", "data": "revealed"}

Remediation:

- Implement strict input validation and sanitization for all user-provided data.
- Use parameterized queries or safe APIs to prevent command injection attacks.
- Enforce least privilege access controls and restrict endpoint usage to authorized users only.

Recommended Code Fix:

```
// Example secure code snippet (Python) using parameterized query
```

SCAN TIMELINE

[Orchestrator] TARGET_ACQUIRED - 2026-03-04 17:31:04

[Sigma] JOB_ASSIGNED - 2026-03-04 17:31:04

[Gamma] LOG - 2026-03-04 17:31:04

[Beta] LOG - 2026-03-04 17:31:04

[Alpha] LOG - 2026-03-04 17:31:04

[Kappa] LOG - 2026-03-04 17:31:05

[Beta] LOG - 2026-03-04 17:31:05

[Alpha] LOG - 2026-03-04 17:31:05

[Beta] LOG - 2026-03-04 17:31:05

[Beta] LOG - 2026-03-04 17:31:05

[Gamma] LOG - 2026-03-04 17:31:05

[Kappa] LOG - 2026-03-04 17:31:05

[Kappa] LOG - 2026-03-04 17:31:05

[Gamma] LOG - 2026-03-04 17:31:05

[Beta] LOG - 2026-03-04 17:31:05

[Alpha] LOG - 2026-03-04 17:31:06

[Alpha] LOG - 2026-03-04 17:31:06

[Gamma] VULN_CONFIRMED - 2026-03-04 17:31:06

[Gamma] VULN_CONFIRMED - 2026-03-04 17:31:06

[Gamma] VULN_CONFIRMED - 2026-03-04 17:31:06

[Kappa] GI5_LOG - 2026-03-04 17:31:14